AFM244 Week 11 - Time Series Forecasting

AFM244 — Time Series Forecasting with Regression

Week 11 (Thursday) — Course Notebook Summary

Overview

This notebook walks through how to use **linear regression as a time series forecasting tool**, using Apple's quarterly sales data as the working example. The core idea introduced is that time series forecasting can be framed as an ordinary least squares (OLS) regression problem, where **time itself becomes an independent variable** — similar to using Time 1, Time 2, Time 3, etc., in accounting and finance.

The notebook progresses from a simple one-variable time trend model to a more advanced model that incorporates a seasonal dummy variable and an interaction term, and finishes by showing how to forecast completely new, out-of-sample future periods.

Requirements

The notebook relies on the following Python libraries:

- `pandas`
- `numpy`
- `statsmodels` (specifically `statsmodels.api` for OLS regression)
- `matplotlib` (for plotting)

It also expects two data files in the working directory:

- `qSales_2024.csv` — quarterly sales data (multiple tickers, including `AAPL`)
- `synthetic_data.csv` — synthetic future period data used for out-of-sample forecasting

Workflow

1. Load and Prepare the Data

- Import the quarterly sales dataset and filter it down to Apple (`AAPL`) records.
- Convert the `datadate` column to a proper datetime type, which matters for time series regressions.

2. Visualize the Data

- Always visualize first: plot Apple's quarterly revenue (`saleq`) over time to inspect the trend before modeling.

3. Build a Time Variable

- Create a `time` column that simply counts sequential periods (1, 2, 3, ...), mirroring how "Time 1, Time 2..." is used elsewhere in finance and accounting.

4. Split into Training and Testing Sets

- Split the data chronologically: the first 75% of observations become the training set, and the final 25% become the testing set (important for time series — the split is not random).

5. Fit a Simple Time-Trend Regression

- Regress revenue (`saleq`) on `time` (plus a constant) using `sm.OLS`.
- Resulting fitted model:

  apple revenue = -13,536 + 1,077 × time

6. Forecast the Test Set

- Use the fitted model to predict revenue for the testing set.
- Use `get_prediction(...).summary_frame(alpha=...)` to generate prediction **intervals**, not just point estimates.
  - `alpha=0.2` → 80% confidence level
  - `alpha=0.1` → 90% confidence level
  - Note: pay attention to `obs_ci_lower` / `obs_ci_upper`, not `mean_se`.

7. Add a Seasonal Dummy Variable

- Create `release_dummy_variable`: a binary indicator equal to 1 when `fqtr == 1` (Apple's fiscal Q1, tied to new product releases), 0 otherwise.
- Create `release_dummy_interaction`: the product of `time` and `release_dummy_variable`, allowing the trend slope to differ during release quarters.

8. Refit the Model with Seasonality

- Re-split the updated dataset into training/testing sets.
- Regress `saleq` on `time`, `release_dummy_variable`, and `release_dummy_interaction`.
- Resulting fitted model:

  apple revenue = -11,044 + 933 × time + (-10,422) × release_dummy_variable + 578 × release_dummy_interaction

  where `release_dummy_interaction = time × release_dummy_variable`.

9. Forecast the Test Set (Updated Model)

- Generate predictions and 80% prediction intervals on the testing set using the expanded model.

10. Forecast Genuinely Future Periods

To forecast values *beyond* the existing dataset (true future forecasting):

1. **Create a synthetic dataset** representing future time periods, along with their corresponding `release_dummy_variable` and `release_dummy_interaction` values.
2. **Apply the exact same prediction steps** used for the testing set, but on the synthetic data instead.

- The notebook loads `synthetic_data.csv` and generates forecasts (with 80% prediction intervals) for these future periods using the fitted seasonal model.

Key Takeaways

- Time series forecasting can be implemented as a regression where time (and functions of time) are treated as independent variables.
- Always visualize data before modeling.
- Train/test splits for time series should preserve chronological order rather than being randomized.
- Dummy variables (e.g., for a recurring seasonal event) and their interaction with time allow the model to capture seasonal effects and shifts in trend.
- Prediction intervals (via `get_prediction().summary_frame()`) provide a *range* of likely outcomes, which is more informative than a single point forecast.
- Forecasting true future values requires constructing a synthetic dataset of future explanatory variables and feeding it through the same trained model.